EUUI--API处理

初始化

EUUIKit.Initialize();　　// 游戏启动时调用一次，自动创建 UIRoot/Camera/EventSystem

---

单人面板管理

OpenAsync<T>(data?) 打开面板（不记录历史，适合弹窗）
Close<T>() 关闭指定面板
CloseAll()　关闭所有面板
CloseAllExcept<T>()　关闭除指定面板外的所有面板
OpenExclusiveAsync<T>(data?)　独占式打开（关闭其他所有面板）
GetPanel<T>() 获取当前激活的面板实例

面板栈导航（主流程页面）

C#

```
1 await EUUIKit.NavigateToAsync<T>();     // 导航到新面板（隐藏当前，记录历史）
2 await EUUIKit.BackAsync();              // 返回上一个面板
3 await EUUIKit.BackToAsync<T>();         // 返回到指定面板（关闭中间所有）
4 await EUUIKit.ClearHistoryAsync();      // 清空历史栈
```

面板状态查询

C#

```
1 EUUIKit.IsPanelOpen<T>()              // 面板是否已打开
2 EUUIKit.IsPanelInStack<T>()           // 面板是否在导航栈中
3 EUUIKit.IsPanelCached<T>()            // 面板是否在 LRU 缓存中
4 EUUIKit.GetCurrentPanelName()         // 当前栈顶面板名称
5 EUUIKit.GetHistoryCount()             // 导航历史深度
6 EUUIKit.GetCachedPanelCount()         // 当前缓存数量
```

多人模式

初始化与切换：

```C#
EUUIKit.InitMultiplayerEventSystem()                                // 默认 4 人 Grid 布局
EUUIKit.InitMultiplayerEventSystem(playerCount, layout, axis?)   // 自定义人数/布局
EUUIKit.EnterMultiplayerMode(playerCount, layout, axis?)         // 进入多人（禁用单人 ES）
EUUIKit.ExitMultiplayerMode()                                    // 退出多人（恢复单人 ES）
EUUIKit.SetMultiplayerLayout(playerCount, layout, axis?)         // 仅切换布局
EUUIKit.IsMultiplayerMode                                        // 当前是否多人模式
```

进入多人模式的步骤 直接调用EnterMultiplayerMode 和分配设备 AssignDefaultDevices即可进入多人模式

```C#
EUUIKit.EnterMultiplayerMode(playerCount,layout,axis?);
EUUIKit.AssignDefaultDevices();
EUUIKit.OpenForPlayerAsync<WndTestMultiPlayer>(index_1).Forget();
EUUIKit.OpenForPlayerAsync<WndTestMultiPlayer>(index_2).Forget();
```

多人面板管理

```C#
await EUUIKit.OpenForPlayerAsync<T>(playerIndex, data?)   // 为指定玩家打开面板
EUUIKit.CloseForPlayer<T>(playerIndex)                    // 关闭指定玩家的面板
EUUIKit.CloseAllForPlayer(playerIndex)                    // 关闭指定玩家所有面板

await EUUIKit.NavigateForPlayerAsync<T>(playerIndex, data?)  // 玩家面板栈导航
await EUUIKit.BackForPlayerAsync(playerIndex)                // 玩家返回上一面板
```

```
7 await EUUIKit.BackForPlayerToAsync<T>(playerIndex)            // 玩家返回到指定面板
8 await EUUIKit.ClearPlayerHistoryAsync(playerIndex)            // 清空玩家历史栈
```

多人的面板状态查询

C#

```
1 EUUIKit.GetMultiplayerRoot(index)          // 获取玩家根节点
2 EUUIKit.GetMultiplayerEventSystem(index)   // 获取玩家 EventSystem
3 EUUIKit.GetMultiplayerPlayerAsset(index)   // 获取玩家 InputActionAsset
4 EUUIKit.GetPlayerLayer(playerIndex, layer) // 获取玩家指定层级
5 EUUIKit.GetPlayerPanel<T>(playerIndex)     // 获取玩家当前面板实例
6 EUUIKit.IsPanelOpenForPlayer<T>(index)     // 玩家面板是否已打开
7 EUUIKit.GetPlayerHistoryCount(index)       // 玩家历史栈深度
8 EUUIKit.GetPlayerCurrentPanelName(index)   // 玩家当前栈顶面板名
9 EUUIKit.MultiplayerPlayerCount             // 当前玩家数量
```

单人多人重设导航焦点

C#

```
1 this.OwnerEventSystem.SetSelectedGameObject(gameobject);
```

使用实例

GameBoot打开第一个面板

C#

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using Cysharp.Threading.Tasks;
4 using EUFramework.Extension.EUInputControllerKit;
5 using EUFramework.Extension.EURes;
6 using EUFramework.Extension.EUUI;
7 using Game.UI;
```

```
using UnityEngine;
using EUFramework.Extension.EUInputControllerKit.MonoComponent;
public class GameBoot : MonoBehaviour
{
    void Awake()
    {
        int _maxPlayerInputControllers = EUInputController.MaxPlayerInputControllers;
    }
    // Start is called before the first frame update
    void Start()
    {
        InitFrameworkAsync().Forget();
    }
    private async UniTaskVoid InitFrameworkAsync()
    {
        bool result = await EUResKit.InitializeAllPackagesAsync();
        if (!result)
        {
            Debug.LogError("资源初始化失败");
            return;
        }
        EUUIKit.Initialize(this.gameObject);
        if (result)
        {
            var p1 = new EUPlayerInputController(); p1.Enable();
            var p2 = new EUPlayerInputController(); p2.Enable();
            // var p3 = new EUPlayerInputController(); p3.Enable();
            // var p4 = new EUPlayerInputController(); p4.Enable();
            await EUUIKit.OpenAsync<WndLoading>();
        }
    }
}
```

WndLoading中打开多人模式

C#

```
//-----------------------------------------------------------------------
// 业务逻辑部分 - 仅在初始创建时生成，请在此编写你的 UI 逻辑
//-----------------------------------------------------------------------
using System.Collections.Generic;
using Cysharp.Threading.Tasks;
using EUFramework.Extension.EUUI;
```

```
7 using EUUI.Extension;
8 using UnityEngine;
9 using UnityEngine.EventSystems;
10 using UnityEngine.UI;
11 namespace Game.UI
12 {
13     public partial class WndLoading : EUUIPanelBase<WndLoading>
14     {
15         public override string PackageName => "Login";
16         public override string PanelName => "WndLoading";
17         public override bool OnCanOpen() => true;
18         #region 业务
19         protected override void OnOpen()
20         {
21             RegisterUIEvent();
22             RegisterEvent();
23         }
24         protected override void OnShow()
25         {
26             var loadingList = LoadingList.GetComponent<LoadingListAdapter>
   ();
27             this.BindSpriteLoader(loadingList);
28             var testData = new List<LoadingListData>();
29             for (int i = 0; i < 10; i++)
30             {
31                 testData.Add(new LoadingListData()
32                 {
33                     Title = "Test " + i
34                 });
35             }
36             loadingList.SetData(testData);
37             loadingList.InputBridge.SetEventSystem(this.OwnerEventSystem);
38             // this.AddClick(Start, () =>
39             // {
40             //     loadingList.InputBridge.EnterList(true);
41             // });
42         }
43         protected override Selectable GetDefaultSelectable()
44         {
45             return this.Start;
46         }
47         protected override void OnHide()
48         {
49         }
50         protected override void OnClose()
51         {
52             UnRegisterUIEvent();
53             UnRegisterEvent();
```

```
        }
        #endregion
        #region 事件
        private void RegisterUIEvent()
        {
            this.AddClick(Start, () =>
            {
                Debug.Log("Start click ");
                EUUIKit.Close<WndLoading>();
                EUUIKit.EnterMultiplayerMode(2, MultiplayerLayoutMode.Linear);
                EUUIKit.AssignDefaultDevices();
                EUUIKit.OpenForPlayerAsync<WndTestMultiPlayer>(0).Forget();
                EUUIKit.OpenForPlayerAsync<WndTestMultiPlayer>(1).Forget();
            });
            var loadingList = LoadingList.GetComponent<LoadingListAdapter>();
            loadingList.InputBridge.OnItemSubmitted = (index) =>
            {
                Debug.Log("Item submitted: " + index);
            };
            loadingList.InputBridge.OnExitList = () =>
            {
                Debug.Log("Exit list");
                //回到面板焦点
this.OwnerEventSystem.SetSelectedGameObject(Start.gameObject);
            };
            loadingList.InputBridge.OnItemFocused = (index) =>
            {
                Debug.Log("Item focused: " + index);
            };
        }
        private void RegisterEvent()
        {
        }
        private void UnRegisterUIEvent()
        {
            var loadingList = LoadingList.GetComponent<LoadingListAdapter>();
            loadingList.InputBridge.OnItemSubmitted=null;
            loadingList.InputBridge.OnExitList = null;
            loadingList.InputBridge.OnItemFocused = null;
        }
        private void UnRegisterEvent()
        {
```

```
        }
        #endregion
    }
}
```

WndTestMultiPlayer 中多人导航

C#

```
//-----------------------------------------------------------------------------
// 业务逻辑部分 - 仅在初始创建时生成，请在此编写你的 UI 逻辑
//-----------------------------------------------------------------------------
using UnityEngine;
using Cysharp.Threading.Tasks;
using EUFramework.Extension.EUUI;
using System.Collections.Generic;
using UnityEngine.EventSystems;
using EUUI.Extension;
namespace Game.UI
{
    public partial class WndTestMultiPlayer : EUUIPanelBase<WndTestMultiPlayer>
    {
        public override string PackageName => "TestMultiPlayer";
        public override string PanelName => "WndTestMultiPlayer";
        public override bool OnCanOpen() => true;
        #region 业务
        protected override void OnOpen()
        {
            RegisterUIEvent();
            RegisterEvent();
        }
        protected override void OnShow()
        {
            Debug.Log("OnShow OwnerPlayerIndex: " + OwnerPlayerIndex);
            Text.text = "OwnerPlayerIndex: " + OwnerPlayerIndex;
            EUUIKit.SetPlayerSelection(OwnerPlayerIndex, Btn_1);
            var loadingList = LoadingList.GetComponent<LoadingListAdapter>();
            this.BindSpriteLoader(loadingList);
            var testData = new List<LoadingListData>();
            for (int i = 0; i < 10; i++)
            {
                testData.Add(new LoadingListData()
```

```
                {
                    Title = "Test " + i
                });
            }
            loadingList.SetData(testData);
            LoadingList.gameObject.SetActive(true);
            Debug.Log($"Name: {loadingList.InputBridge.name} -- {loadingList.name}");
            loadingList.InputBridge.SetEventSystem(this.OwnerEventSystem);
        }
        protected override void OnHide()
        {
        }
        protected override void OnClose()
        {
            UnRegisterUIEvent();
            UnRegisterEvent();
        }
        #endregion
        #region 事件
        private void RegisterUIEvent()
        {
            var loadingList = LoadingList.GetComponent<LoadingListAdapter>();
            this.AddClick(Btn_1, () =>
            {
                loadingList.InputBridge.EnterList(true);
            });
            loadingList.InputBridge.OnItemSubmitted = (index) =>
            {
                Debug.Log("Item submitted: " + index);
            };
            loadingList.InputBridge.OnExitList = () =>
            {
                Debug.Log("Exit list");
                //回到面板焦点
this.OwnerEventSystem.SetSelectedGameObject(Btn_1.gameObject);
            };
            loadingList.InputBridge.OnItemFocused = (index) =>
            {
                Debug.Log("Item focused: " + index);
            };
        }
        private void RegisterEvent()
        {
        }
        private void UnRegisterUIEvent()
```

```
        {
            var loadingList = LoadingList.GetComponent<LoadingListAdapter>();
            loadingList.InputBridge.OnItemSubmitted = null;
            loadingList.InputBridge.OnExitList      = null;
            loadingList.InputBridge.OnItemFocused   = null;
        }
        private void UnRegisterEvent()
        {
        }
        #endregion
    }
}
```